

Dungeon Crawler 2D

A 2D Grid-Based Dungeon Game in Java & JavaFX

Tech Stack: Java 23 | JavaFX 21 | Apache Maven | MVC Architecture

Overview

Dungeon Crawler 2D is a dynamic, turn-based 2D dungeon exploration game built using Java and JavaFX. The application strictly follows the **Model-View-Controller (MVC)** and **Observer** design patterns, ensuring a clean decoupling of game logic, spatial algorithms, and board state from user interface presentation and input processing.

Players navigate a procedurally populated dungeon floor filled with obstacles, hostile adversaries, and hidden loot. The objective is to gather valuable treasure, recover bonus hearts, outwit roaming enemies, and reach the exit portal to descend into deeper, increasingly dangerous dungeon chambers.

Key Features

- **Procedural Level Generation:** Randomizes placement of walls, treasures, hearts, enemies, exit portal, and player hero while strictly guaranteeing grid boundary integrity.
- **Dual-Mode Turn-Based Enemy AI:**
 - *Easy Mode:* Enemies pick non-colliding random directions within dungeon boundaries.
 - *Hard Mode:* Aggressive vector-delta pathing where enemies actively hunt the player's coordinate position.
- **Life & Health System:** Collectible heart pieces grant protective hit points against enemy collisions.
- **Persistent Session Scoring:** Tracks current run points (+50 per gold doubloon) and updates high scores across playthroughs.
- **Integrated Atmospheric Audio:** Continuous dungeon theme soundtrack using JavaFX Media.
- **Decoupled Event Architecture:** The JavaFX view re-renders purely in reaction to observer notifications emitted by the model.

Architecture & Design Patterns

Model-View-Controller (MVC)

- **Model:** Manages board matrix representation, positioning (`Posn`), score, lives, difficulty, and room progression. Encapsulates entity contracts (`Piece`, `APiece`, `MovablePiece`) and turn resolution.
- **View:** JavaFX views (`TitleScreenView`, `GameView`, `EndView`) observe the model and render reactive graphics without direct business logic coupling.
- **Controller:** Translates keyboard keystrokes (WASD) and UI button presses into semantic model mutations.

Observer Pattern

The model implements the `Subject` interface, allowing any view implementing `Observer` to register via `addObserver(Observer o)`. State mutations trigger `notifyObservers()`, enabling seamless, decoupled view updates.

Collision State Machine

Movement resolution evaluates interactions between moving entities and target coordinates:

- **Hero & Empty Spot:** Safe movement; turn concludes normally.
- **Hero & Treasure:** Grants +50 points and collects the chest.
- **Hero & Heart:** Increases life points and collects the heart.
- **Hero & Exit:** Advances dungeon floor (`CollisionResult.Result.NEXT_LEVEL`).
- **Hero & Enemy:** Consumes 1 bonus life if available; otherwise triggers `Result.GAME_OVER`.
- **Enemy & Board:** Enemies consume unprotected treasures and damage the hero, but cannot cross walls or exits.

Project Structure

```
src/
|-- main/
|   |-- java/com/comp301/a09dungeon/
|   |   |-- Main.java                # Application entry point
|   |   |-- controller/
|   |   |   |-- Controller.java      # Controller interface
|   |   |   |-- ControllerImpl.java  # Event-handling implementation
|   |   |-- model/
|   |   |   |-- Model.java           # Core model contract
|   |   |   |-- ModelImpl.java       # Game rules and turn lifecycle
|   |   |   |-- Observer.java        # Observer interface
|   |   |   |-- Subject.java         # Subject interface
|   |   |-- board/
|   |   |   |-- Board.java           # Board layout contract
|   |   |   |-- BoardImpl.java       # Grid matrix & movement validation
|   |   |   |-- Posn.java            # Immutable (row, col) coordinates
|   |   |-- pieces/
|   |   |   |-- Piece.java           # Entity contract
|   |   |   |-- APiece.java          # Abstract base entity
|   |   |   |-- MovablePiece.java    # Move/collision interface
|   |   |   |-- CollisionResult.java  # Collision outcome container
|   |   |   |-- Hero.java            # Player avatar
|   |   |   |-- Enemy.java           # Hostile entity
|   |   |   |-- Treasure.java        # Collectible point item
|   |   |   |-- Heart.java           # Collectible health item
|   |   |   |-- Wall.java            # Impassable barrier
|   |   |   |-- Exit.java            # Level transition gate
|   |   |-- view/
|   |   |   |-- AppLauncher.java     # JavaFX Application bootstrapper
|   |   |   |-- FXComponent.java     # View component interface
|   |   |   |-- View.java            # Master view router
|   |   |   |-- TitleScreenView.java # Start / settings menu
|   |   |   |-- GameView.java        # Main dungeon viewport
|   |   |   |-- EndView.java         # Game-over & retry screen
|   |   |   |-- Music.java           # Background audio controller
|   |-- resources/
|   |   |-- *.jpg                   # Sprites (hero, enemy, tiles, UI buttons)
|   |   |-- music.mp3               # Background soundtrack
|   |   |-- style/dungeon.css       # JavaFX styling definitions
pom.xml                             # Maven configuration & plugins
```

Getting Started

Prerequisites

- **Java Development Kit (JDK):** Version 21 or higher (JDK 23 recommended).
- **Apache Maven:** Version 3.8+ installed and available on PATH.

Build and Execution

Compile and launch the game directly via the JavaFX Maven plugin:

```
mvn clean javafx:run
```

Packaging Executable JAR

Build a standalone executable JAR bundling all required dependencies:

```
mvn clean package
java -jar target/a09-dungeon-1.0-SNAPSHOT-jar-with-dependencies.jar
```

How to Play

Controls

Action	Keyboard	On-Screen Button
Move Up	W	Click Up Button (▲)
Move Down	S	Click Down Button (▼)
Move Left	A	Click Left Button (◀)
Move Right	D	Click Right Button (▶)

Game Mechanics

1. **Survive & Advance:** Step into the exit portal to progress to the next room. Each subsequent floor adds more enemies.
2. **Collect Treasure:** Pick up golden chests to boost your session score by 50 points each.
3. **Collect Hearts:** Gathering hearts grants extra life protection against enemy collisions.
4. **Avoid Death:** Colliding with an enemy without extra lives ends the expedition and records your high score.

Difficulty Modes

- **Easy Mode:** Enemies roam randomly across valid adjoining tiles.
- **Hard Mode:** Enemy AI computes vector deltas toward the hero, actively cornering and hunting the player down.